Project Report: Hospital Management System V2

1. Student Details

- **Name:** Harshit Shrivastav
- **Roll Number:** DS23f3003479
- **Email:** 23f3003479@ds.study.iitm.ac.in
- **About Me:** I am a BS degree student at IIT Madras. I really like automating things, like creating bots, writing scripts, and building web applications. I enjoy turning everyday problems into efficient digital solutions.

---

2. Project Details

Project Title: Hospital Management System V2

Problem Statement:

Hospitals need efficient systems to manage patients, doctors, appointments, and treatments. Currently, many hospitals use manual registers or disconnected software, which makes it difficult to manage records, avoid scheduling conflicts, and track patient history. The goal is to build a robust, role-based web application that digitizes these core workflows.

Approach:

- Developed a decoupled architecture using **Flask** (REST API Backend) and **VueJS with Vite** (Modern Frontend SPA).
- Implemented **Role-Based Access Control (RBAC)** for Admin, Doctor, and Patient using JWT tokens.
- Utilized **Redis** for caching frequently accessed data (e.g., doctor availability) and as a message broker for **Celery**.
- Integrated **Celery** for handling asynchronous tasks such as monthly report generation and CSV exports to prevent API blocking.
- Ensured database schema is created **programmatically** using SQLAlchemy models.
- Built a responsive UI using **Bootstrap 5** integrated within Vue components.

---

3. AI / LLM Declaration

I used ChatGPT (GPT-5) to assist in:

- Debugging VueJS reactive state issues and Flask-CORS configuration.

- Understanding Celery task routing and Redis integration patterns.
- Formatting HTML for email templates used in monthly reports.

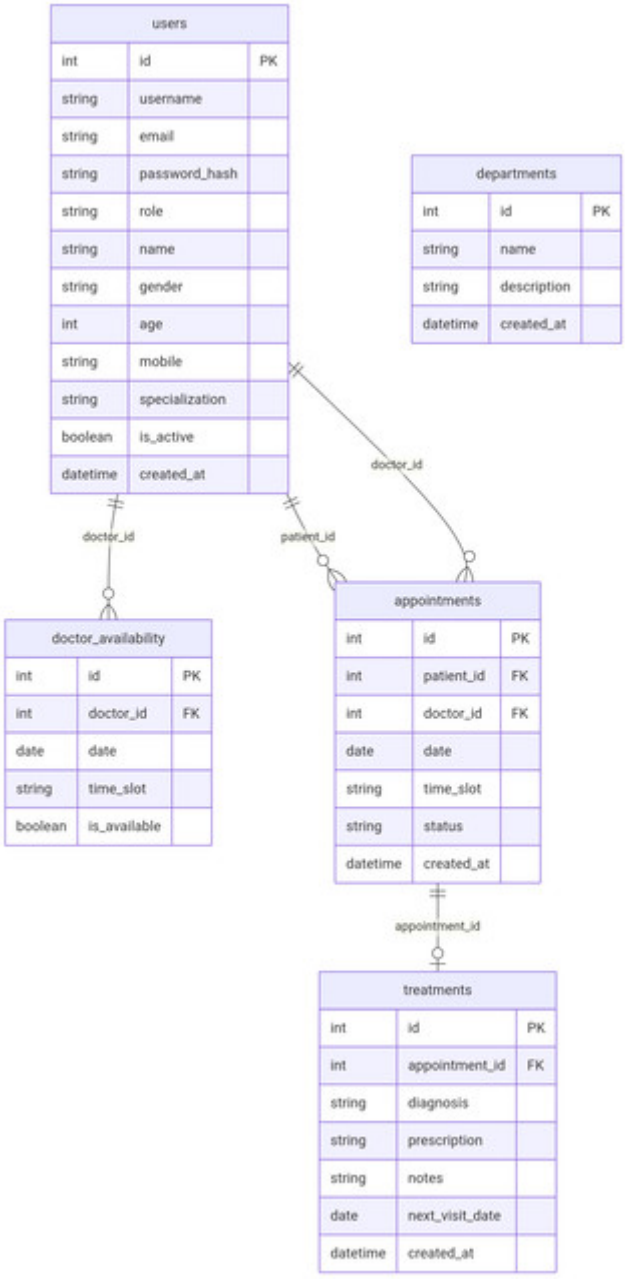
Extent of AI/LLM usage: ≈ 8 - 10%

All final implementation logic, database modeling, security configurations, and feature integration were performed manually.

---

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend REST API framework
VueJS 3 + Vite	Modern, reactive frontend UI (Single Page Application)
Flask-SQLAlchemy	ORM for programmatic SQLite database interactions
Flask-JWT-Extended	Token-based authentication and role protection
Werkzeug	Password hashing and security utilities
SQLite	Lightweight local database (`hms.db`)
Redis	Caching layer and Celery message broker (Upstash)
Celery	Asynchronous task queue for batch jobs
Bootstrap 5	Responsive frontend styling framework
Chart.js	Visualization of stats on dashboards



ER Diagram Preview:

5. Database Schema / ER Diagram

Tables:

1. **users** - Stores all user accounts (Admin, Doctor, Patient)
    - `id`, `username`, `email`, `password_hash`, `role`, `name`, `gender`, `age`, `experience`, `dept` (department), `mobile`, `created_at`
2. **doctor_department** - Master list of medical specializations
    - `id`, `depts` (department name)
3. **doctor_availability** - Tracks weekly availability (Next 7 days)
    - `id`, `doctor_id` (FK), `date`, `time_slot` (Morning/Evening), `is_available`
4. **appointments** - Manages patient-doctor bookings
    - `id`, `patient_id` (FK), `doctor_id` (FK), `date`, `time_slot`, `status` (booked/completed/cancelled), `booking_date`, `doc_dept`, `doc_name`

5. **patient_history** - Clinical records linked to completed appointments
    - `id`, `appointment_id` (FK), `patient_id` (FK), `doctor_id` (FK), `visit_type`, `test_done`, `diagnosis`, `prescription`, `medicines`, `visit_date`

Relationships: - User (1:N) → Appointments (as patient) - User (1:N) → Appointments (as doctor) - User (1:N) → DoctorAvailability - User (1:N) → PatientHistory - Appointment (1:1) → PatientHistory

---

6. Architecture and Features

Architecture Overview:

```
hms-v2/
├── backend/                  # Flask application
│   ├── app/
│   │   ├── models/       # SQLAlchemy schema definitions
│   │   ├── routes/       # REST API endpoints
│   │   ├── services/    # Business logic
│   │   ├── tasks.py     # Celery workers
│   │   └── celery_app.py
│   ├── config.py
│   └── main.py
└── frontend/
    └── hms-v2/           # VueJS application
        └── src/
            ├── views/    # Role-based components
            ├── services/# API service layer
            └── router/   # Vue Router config
```

Core Features & Role Functionalities:

Authentication & Security

- JWT Token-based authentication (Flask-JWT-Extended)
- Programmatic creation of Admin account on database initialization
- Role-protected API endpoints (`@jwt_required()`)
- Cookie-based JWT storage with CORS support

Admin Capabilities

- Dashboard with statistics (Total Doctors, Patients, Appointments)
- CRUD operations on Doctor profiles
- Department management (create/list)
- View pending and completed appointments
- Block/unblock users (soft-delete)

Doctor Capabilities

- Manage availability for the next 7 days (Morning/Evening slots)

- View assigned appointments filtered by date
- Update appointment status to Completed
- Log Treatment notes (Diagnosis, Prescription, Medicines)

Patient Capabilities

- Self-registration and profile management
- Search for doctors by department
- View 7-day availability calendar
- Book, Reschedule, or Cancel appointments
- View complete treatment history

Backend Batch Jobs (Celery + Redis)

- **User-Triggered Export**: Patient can request a CSV export of their full treatment history

Performance Optimizations

- **Redis Caching**: Doctor availability cached to reduce DB reads
- Cache invalidates on booking or availability update

---

7. API Endpoints

Method	Endpoint	Role	Description
POST	`/api/register`	Public	Patient registration
POST	`/api/login`	Public	User login (returns JWT cookie)
POST	`/api/logout`	Public	User logout
GET	`/api/login-status`	Public	Check authentication status
GET	`/api/admin/dashboard`	Admin	Dashboard statistics
POST		Admin	

Method	Endpoint	Role	Description
	`/api/admin/create-department`		Create new department
GET	`/api/admin/get-departments`	Admin	List all departments
POST	`/api/admin/create-doctor`	Admin	Create doctor account
GET	`/api/doctors`	Public	List all doctors
GET	`/api/doctors/<id>`	Public	Get doctor details
GET	`/api/doctor/availability`	Doctor	Get doctor’s availability
POST	`/api/doctor/availability`	Doctor	Set availability
GET	`/api/doctor/dashboard`	Doctor	Doctor’s dashboard
POST	`/api/doctor/appointment/<id>/complete`	Doctor	Complete appointment
POST	`/api/doctor/patient-history`	Doctor	Add treatment notes
GET	`/api/patient`	Patient	Patient dashboard
GET	`/api/patient/profile`	Patient	Get patient profile

Method	Endpoint	Role	Description
PUT	`/api/patient/profile`	Patient	Update patient profile
GET	`/api/patient/departments`	Patient	List departments
GET	`/api/patient/doctors/<dept_id>`	Patient	Doctors by department
GET	`/api/patient/doctor/<id>/slots`	Patient	Doctor’s available slots
POST	`/api/patient/book-appointment`	Patient	Book appointment
PUT	`/api/appointments/<id>`	Patient	Reschedule appointment
POST	`/api/cancel-appointment`	Patient	Cancel appointment
GET	`/api/patient/my-history`	Patient	Patient treatment history
POST	`/api/patient/export`	Patient	Trigger CSV export

---

8. Running the Project

Backend

```
cd backend
pip install -r requirements.txt
python main.py
```

Frontend

```
cd frontend/hms-v2
npm install
npm run dev
```

Celery Worker (for async tasks)

```
cd backend
celery -A app.celery_app worker --loglevel=info
```

---

9. Default Credentials

- **Admin Email:** admin@hms.com
- **Admin Password:** admin@hms.com
